This is the author's version (manuscript) of the work that was accepted for publication in AIAA SCITECH 2025 Forum.
The Version of Record is available online at https://arc.aiaa.org/doi/10.2514/6.2025-1791 .

Evolving AI-Driven Workflow Management, Part B: Non-unique Engineering Workflows and Scalable Open-weight Agents

Nicolai Forsthofer * and Oliver Kunc†
German Aerospace Center, Stuttgart, Germany

Workflow engines play an important role for modern engineering, especially when the complexity of the subject is high. Large Language Models could potentially provide a powerful user interface, if they can be set up to reliably transform natural language inputs into correct workflow instructions. Previous works investigated this possibility with a single proprietary Large Language Model as the only involved AI system. The accompanying part A of the current work enhances that method by implementing a multi-agent architecture of proprietary models and Open Weight Models resulting in reduced context window sizes, and by also incorporating a data provenance system. The present part B addresses the real-world problem of ambiguity of workflows and how to solve this problem in a scalable manner. The main contributions are twofold. First, the non-uniqueness of results of queries to knowledge graphs of realistic workflows for computational engineering is classified as either "by multi-fidelity" or "by redundancy". Second, it is shown that LLMs with large context window can be capable of resolving such non-uniqueness whereas fine-tuned Small Language Models contribute in other ways to the scalability of the multi-agent system of part A.

I. Nomenclature

AI	=	Artificial Intelligence
API	=	Application Programming Interface
FT	=	Fine Tuning
GTlab	=	Gas Turbine Laboratory
JSON	=	JavaScript Object Notation
LLM	=	Large Language Model
KG	=	Knowledge Graph
RAG	=	Retrieval Augmented Generation
SLM	=	Small Language Model
SysML	=	Systems Modeling Language
WfMS	=	Workflow Management System

II. Introduction

Holistic engineering platforms provide solutions to create, edit, and manage workflows. Such functionality is called a Workflow Management System (WfMS) in this paper. Depending on the chosen approach for the WfMS, the user experience may differ significantly. On highly complex platforms, even the issue of providing the user an easily digestible overview of all (or just all relevant) process elements is non-trivial. In the scope of the current work, the suitability of an assistance by means of AI is investigated. More specifically, Large Language Models (LLMs) are to be employed as a user interface to the workflow engine. The main reasons to do so are:

1) Productivity: LLM assistance can accelerate the successful interaction of users with the WfMS.
2) Accessibility: the astonishing LLM capabilities of natural language processing can make expert processes accessible to non-experts.

*Researcher, Institute of Structures and Design, Nicolai.Forsthofer@dlr.de
†Researcher, Institute of Structures and Design, Oliver.Kunc@dlr.de

3) Knowledge management: the maintenance and transfer of expert knowledge within organization is of great importance and would benefit from an AI-assisted WfMS.

Probably the most prominent example for productive general natural language processing capability is the utilization of LLMs for programming assistance [1]. A more workflow-based approach to coding was presented in [2]. There, API-calling coding tasks have been realized with a classical retrieval-augmented generation (RAG) approach with API documentations being referenced in a vector database. More general tasks in the context of state-defined workflows have been shown to be realizable with LLMs using prompt-engineering in [3]. Various domain-specific solutions have been proposed, e.g. in [4] where API-calling workflows in finance have been realized by means of LLMs with, too, prompt-engineering. In the field of design engineering, high-level architectures of AI-assisted engineering systems have been reviewed in [5].

These and related works reveal interesting approaches to the overarching goal of automated and user-friendly workflow creation for specific use cases or contexts. The present series of papers focuses on computational engineering workflows in a complex, multi-disciplinary, and collaborative setting.

In the preceding work [6], a straight-forward application of LLMs at the interface of the user and the workflow engine of a design platform has been studied. The results have been promising and have encouraged the continuation of research in this direction. In part A [7] of the present two-part series of works, the procedure originally developed in [6] is enhanced by multiple aspects. For one, the problems of limited context window size and varying distribution of attention within that window is addressed by means of a multi-agent system equipped with a classical, context-enriching RAG scheme. The resulting AI-driven system is more reliable and performant due to specialized sub-systems.

In the present Part B, the focus is placed on a practical problem with an example application drawn from computational structural mechanics processes for jet engine development, particularly addressing real-world engineering workflows encountered in both industry and academia: the ambiguity of possible workflows. The guiding question of this paper is:

"How can we solve the problem of non-uniqueness in a scalable way?"

An AI system is *scalable* if its architecture does not need to change substantially even if its context or environment becomes drastically more complex. For instance, if it needs to interact with 100 times the amount of external data or processes, it should not require major adaptions. Also, in such a case the setup pipeline (data generation and pre-processing, training, evaluation) must remain very similar. In other words, scalability of an AI system means that the system as-is keeps much of its usefulness in a scaled environment. The obvious next question is, how to achieve a scalable AI system?

A key factor of scalability is *efficiency*. In the context of LLMs, every decrease of the utilized context window length and any savings in model memory size is a contribution to efficiency, provided that the quality of the responses does not deteriorate. Efficiency is always recommended, even if certain individual use cases can be covered by less efficient methods. Efficiency of LLMs directly manifests in many concrete ways, each of which itself is a contribution to scalability. For instance, a fixed hardware setup hosting a more efficient LLM system could

- have reduced latency,
- handle of more queries per unit of time,
- return higher-quality responses, or
- incorporate larger amounts of dynamically added data.

In essence, scalability is a prerequisite for operational sustainability and efficiency heavily contributes to scalability.

III. Methodology and data

A. State of the art

1. Generic Workflows

The foundation for this work is built on the findings of a previous study that explored the use of LLMs for generating engineering workflows, [6]. In that study, workflows have been constructed for a dataset of four generic examples, each with a unique solution, cf. Figure 1. Users have provided both precise inputs and outputs and the LLM has proposed a corresponding workflow. While effective, the approach is limited to cases where a single, unambiguous solution existed.

Part A of this two-part series addresses challenges posed by context window limitations in LLMs when handling more complex workflows. By leveraging function calling and multi-agent systems, the authors demonstrate that even smaller-scale models, such as Gemma or LLaMA, could generate workflows effectively. They introduce a graph-based structure to represent workflows, enabling pathfinding algorithms to identify optimal solutions. This approach is

integrated into the GTlab framework (a collaborative engineering software platform, [8]), ensuring compatibility with existing engineering tools. However, the focus remains on workflows with unique solutions.

This paper builds upon these advancements by addressing the limitations of the unique solution assumption. While the methods in part A are effective for constrained examples, real-world engineering workflows often exhibit non-uniqueness. These complexities necessitate a more adaptive approach, which is explored in the following section.

2. Non-unique solutions in real-world tool landscapes

In real-world engineering workflows, non-uniqueness arises from two primary factors: tool redundancy and multi-fidelity options. Tool *redundancy* occurs when multiple tools perform the same function within a workflow. For example, two mesh generators may produce identical or interchangeable finite element meshes for the same geometry, leading to ambiguity in selecting the optimal path in the landscape of tools.

Multi-fidelity options further complicate workflow generation. Consider the design of turbine disks: a parametric method may provide quick but inaccurate approximations, whereas an optimization-based approach may yield highly detailed results at the cost of computational time. As a third, intermediate option, a surrogate model based on machine learning methods may balance speed and accuracy, yet only within its training range. Each method has distinct advantages and trade-offs, making it challenging to determine the most suitable tool for a given context. Figure 2 shows two examples of ambiguous workflows.

To address these challenges, we propose leveraging the decision-making capabilities of LLMs. By integrating contextual information, the LLM can evaluate the trade-offs between tools and recommend the most appropriate path through the workflow graph. This approach extends the software infrastructure developed in part A and can be combined with multi-agents, enabling it to handle the inherent complexity of real-world workflows. Whereas multiple LLMs ought to be investigated for their suitability for this purpose, the most ambitious undertaking of this work is to evaluate the respective capabilities of a fine-tuned Small Language Model (SLM).

B. Model Selection

Starting with the release of Llama by META, a vast amount of open-weight models and fine-tunes were released up to date, from which a selection of promising models must be made. To quantify the abilities of a model, various established benchmarks are available. When selecting benchmarks for evaluating LLMs in specific tasks, it is crucial to consider the benchmark relevance and the difficulty level. The specific task of workflow generation resembles a programming task where a natural language prompt should lead to a specific language syntax describing process elements as well as their inputs and outputs. The most promising benchmarks therefore are the ones that best evaluate logic reasoning, follow direct instructions, and exhibit strong programming capabilities. Established benchmarks for these use cases include:

- HellaSwag [9]: A test of commonsense inference, which is easy for humans (95%) but challenging for state-of-the-art models.

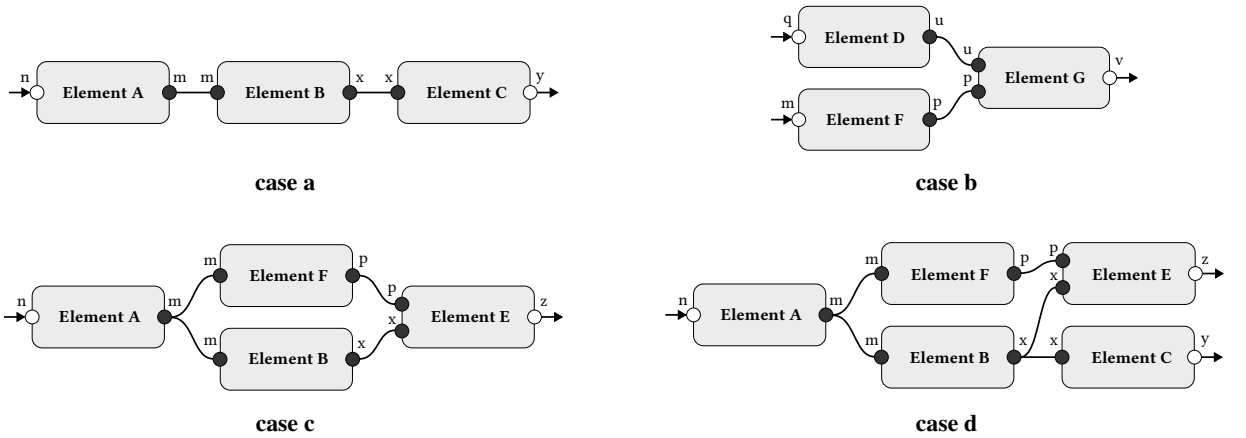


Fig. 1 Generic unique workflow examples from [6]

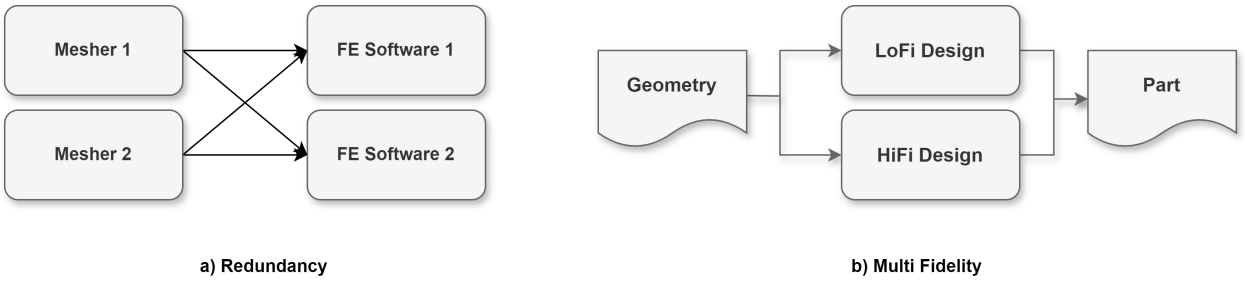


Fig. 2 Categories for ambiguity in engineering workflows

- MBPP [10]: A set of introductory programming problems in Python that is useful for evaluating an LLM's ability to handle fundamental coding tasks that a novice programmer would encounter.
- GSM8k [11]: Diverse grade school math problems in text form to measure a model's ability to solve multi-step mathematical reasoning problems.

A combination of these benchmarks should lead to a well-rounded view of an LLM's performance for the workflow generation task.

By considering these factors and by also taking into account the hardware requirements to finetune LLMs, the language model Phi-3.5-mini-instruct [12] was chosen for this work. With its "only" 3.82 billion parameters, it falls into the subcategory of SLM. The following models are used to compare the results and get insights into the specific abilities needed to generate real world engineering workflows:

- Llama3-70B-instuct and Llama3-8B-instruct
- Qwen2.5-72B
- ChatGPT-4o

C. Fine-tuning GTlab information (FT-GTlab)

The first step of improving the efficiency of the multi-agent system of part A [7] is to fine-tune the user interface agent. By incorporating knowledge of the software platform GTlab directly into the LLM, it should be possible to effectively reduce the utilized context window length.

FT-GTlab step 1: raw data

The starting point for the collection of training data containing information about GTlab are snippets from the (preliminary) developer documentation linked on the website www.gtlab.de. The collected data contains less than 7,100 characters (approximately 1.5 pages of A4 paper). It consists of natural language describing various aspects of the framework, such as its name, origin, purpose, architecture, modularity, intended use cases, etc. Moreover, information about its systems engineering aspects such as the possibilities of handling diverse data models, versioning, and provenance. Other mentioned aspects include the workflow management system, scripting possibilities, the included CAD kernel, and data analysis tools. Elementary user instructions are also provided, such as how to create, load, save, close, and rename GTlab projects via interaction with the graphical user interface (GUI).

FT-GTlab step 2: inflation of manually curated conversational data

In order to convert and extend this running text into a conversational setting, a commercial LLM is employed. In essence, the approach of synthetic data generation or data augmentation with the help of LLMs is followed, which is known at least in principle since [13] and has been extensively reviewed in [14]. More precisely, the commercial product GPT-4o (*gpt-4o-2024-08-06*) by OpenAI is queried with the following basic settings:

- System prompt: contains the raw data and instructions to provide responses in a strict conversational format, concatenating the entire history into a single conversational history.
- Structured output: JSON schema compatible with the `apply_chat_template` function from the Huggingface Transformers library [15].

With this setup, seven artificial conversations are created. These vary in length between two and seven questions/instructions and answers, totaling 29 pairs of contents by the user and the assistant and more than 25,000 characters.

Next, an automated script is created to further inflate this dataset with the following settings:

- System prompt: contains instructions to reformulate the following conversation, i.e. the contents of both the user and the assistant. Details are given about paraphrasing, avoidance of literal repetitions, formulations of similar questions and similar answers, etc. A few examples of alternative user questions and the intents behind them are also given.
- Structured output: same as above.
- Messages: one entire conversation of the seven conversations that were previously created.

The repeated (trivially in parallel) execution of this script, looping over the original seven conversations, leads to a large dataset. The dataset consists of an ASCII file in JSONL format with each line containing one single, complete conversation. It, however, contains duplicates which need to be removed with a one-line batch command. The final result is a dataset of 1102 artificial conversations with a user asking questions about or requesting information on GTlab, its purpose, and how to use it, and an assistant delivering replies. These conversations contain approximately 2.3 million content characters, i.e. not counting those related to the schema.

FT-GTlab step 3: fine-tuning Phi-3.5

The JSONL file is loaded onto a compute node with four NVIDIA A100 Tensor Core GPUs with 40 GB of RAM. The Huggingface Transformers library is utilized for the supervised fine-tuning task based on the template script [16]. It also includes the libraries Accelerate by Huggingface [17], Deepspeed by Microsoft [18], and PyTorch 2 [19]. Due to limited available time for this case study, this template script is only minimally adjusted until satisfactory results are achieved. A sophisticated hyper-parameter optimization is omitted.

D. Knowledge graph retrieval-augmented generation

The foundation of Graph Retrieval-Augmented Generation (Graph RAG) lies in the construction of a robust knowledge graph that encodes the relationships between tools, data, and their dependencies and was first introduced by Microsoft [20]. Unlike classical RAG, which relies on vector databases for similarity-based retrieval, Graph RAG directly utilizes the structure and properties of a knowledge graph to retrieve relevant information. This distinction allows Graph RAG to better represent complex engineering workflows, where dependencies and multi-step processes are critical.

In a conventional RAG setup, textual or numerical data is embedded into a vector space using a pre-trained embedding model. When a user submits a query, similarity search techniques, such as cosine similarity, are employed to identify the most relevant entries in the vector database. These entries are appended to the prompt and passed to the LLM for generating a response. Graph RAG, on the other hand, replaces the vector database with a graph database. Here, information is stored as nodes and edges, where nodes represent entities (e.g., tools, data points) and edges capture their relationships (e.g., input-output dependencies). In Graph RAG, embeddings are attached as properties of the nodes rather than existing as standalone entities associated with text chunks as visualized in Figure 3.

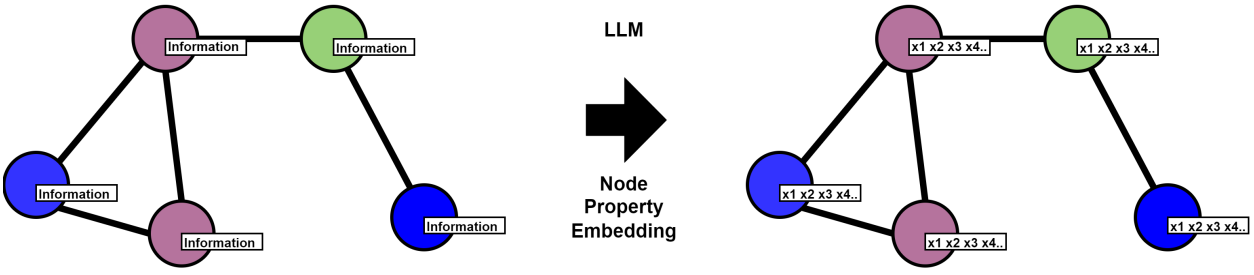


Fig. 3 Graph embedding principle

For instance, a node representing a tool might include properties such as its name, description, input requirements,

and output characteristics. These properties can be embedded into vector representations to facilitate similarity search within the graph. When a user submits a query, the system identifies the most relevant nodes using similarity metrics and retrieves either a single node or a subgraph containing related nodes and edges. This subgraph is then appended to the LLM prompt as natural language, enabling the model to generate responses enriched with contextual information from the graph, cf. Figure 4. The schema of the knowledge graph used for the graph RAG method is designed to capture

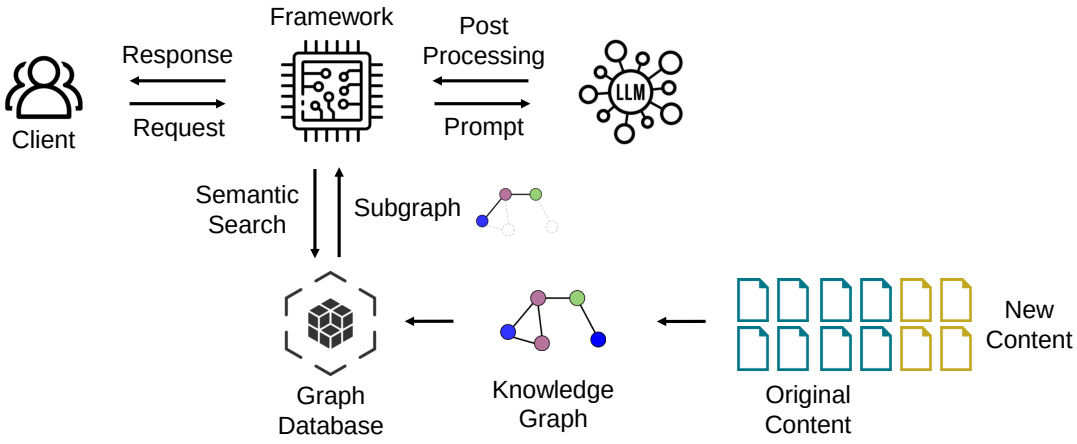


Fig. 4 Graph-RAG Architecture

the essential elements of engineering workflows. In our framework, the graph is constructed using two primary types of nodes:

Tool Nodes

These nodes represent the individual tools or processes available within the engineering framework. Each tool node includes properties such as its name, description, input requirements, and output characteristics. For example, a tool node might describe a finite element mesh generator with inputs like geometry and mesh density parameters and outputs like a finite element mesh.

Data Nodes

These nodes represent data entities, which serve as inputs or outputs for the tools. Data nodes include properties such as their name and description. For instance, a data node might represent a geometry file or a set of eigenfrequency results.

Edge Types

Edges in the graph represent the relationships between tools and data. Specifically, edges are labeled as "INPUT_OF" or "OUTPUT" to indicate the flow of information. An edge might connect a data node representing a geometry description to a tool node representing a mesh generator, labeled as "INPUT_OF." This structure encodes the dependencies between tools but also enables the determination of execution order within workflows.

One of the key challenges in engineering workflows is the presence of non-unique solutions. Multiple tools may perform the same function (tool redundancy), and different fidelity levels (multi-fidelity options) may offer varying trade-offs between speed, accuracy, and memory. A user query to compute the eigenfrequencies of a rotor blade might be addressed using an analytical method, an optimization-based approach, or a machine learning surrogate. Each path through the graph represents a potential workflow, and the choice of path depends on the specific context and requirements. To address this, the knowledge graph enables pathfinding algorithms to identify all possible paths from a starting node (e.g., a data node representing rotor geometry) to a target node (e.g., a data node representing eigenfrequency results). All of these feasible paths are then presented to the LLM, along with the descriptions of the

tools involved. The LLM is tasked with the evaluation of the trade-offs and the selection the most appropriate path based on the query context.

The integration of LLMs into the Graph RAG framework enhances its decision-making capabilities. The detailed descriptions stored in the graph allow the LLM to assess the suitability of different tools and workflows. If a user prioritizes computational efficiency, the LLM might recommend a low-fidelity analytical method and if accuracy is more important, the LLM might suggest a high-fidelity optimization-based approach.

The system architecture allows for high scalability as each node in the graph can be treated as a "black box" or "white box," depending on the level of detail required. In the black-box view, a node represents a single tool or process, abstracting its internal workings. In the white-box view, the node can be expanded to reveal its internal steps, which may themselves form a subgraph. This hierarchical representation allows the framework to scale from simple workflows to highly complex, multi-step processes. To test this hypothesis, a KG with high enough complexity is created. The dynamic scalability of the KG is a critical advantage of the Graph RAG approach. By treating nodes as modular entities, the graph can accommodate new tools and data types without requiring significant restructuring.

In summary, the combination of knowledge graphs, pathfinding algorithms, and LLMs provide a framework for generating adaptive engineering workflows. This adresses the challenges of non-unique solutions and uses the contextual understanding of LLMs, the Graph RAG approach represents a promising method to integrate highly specialized engineering knowledge into the process of generating non-unique workflows.

E. Fine-tuning knowledge graph node descriptions (FT-KG)

After general information on GTlab is incorporated directly into the LLM and information on process elements can be supplied in the prompt rather efficiently by means of the KG-RAG technique, the next step is now to further the efficiency. If the definitions and descriptions of the process elements can also be fine-tuned into the LLM, the required context window length is greatly reduced.

In a productive real-world workflow environment, such an undertaking would be advisable only under certain circumstances. Fine-tuning is a comparatively involved process, in contrast to editing a KG. Whereas the latter can be modified quite dynamically without many dependencies, the former affects the LLM's parameters and potentially the general model performance. Therefore, only well-established process elements that are unlikely to change fundamentally or to vanish in the future are recommended for fine-tuning their definitions and descriptions.

FT-KG step 1: raw data

The raw data for this fine-tuning task consists of the JSON description of the KG function nodes. The following example describes the node representing a contour plotting function:

```
{
  "type": "Function",
  "properties": {
    "Name": "Kontur_Plotter",
    "Input": ["VMAP_Results"],
    "Output": ["KonturPlot"],
    "Description": "Generates a visual representation of stress or displacement
    fields. The color coding is fixed to rainbow. Through visualization peaks in the
    stress distribution can easily be detacted and also the deformation mode of the
    blade can be seen. Is useful for static as well as dynamic analysis types"
  }
}
```

It is obvious that not much care has been taken while creating this node object: there are fragments of other languages than English ("Kontur"), a mixture of naming conventions (class names with or without underscores), plain misspellings ("detacted", *sic*) and, generally speaking, a lack of rigor in the description. This example is representative of the whole set of node definitions.

Such inaccuracies are a deliberate feature of this dataset because the LLM is challenged to handle such uncertainties and brevity. In productive real-world scenarios, it should not be assumed that much effort is being made in the creation and maintenance of such process labels. Robustness with respect to meta-data curation can only be of advantage, lowering the barrier for practical application.

Note that it would be straight-forward to adopt a SysML schema in order to couple this approach with the actual GTlab workflow management system as described in part A [7, III.B.]. Nevertheless, the JSON format was more straight-forward for use in the present feasibility study.

FT-KG step 2: data inflation and consolidation

Similar to section III.C, an inflation pipeline is set up based upon the commercial product GPT-4o and its structured outputs functionality. The setup is as follows:

- System prompt: the entire definition of a function node of the KG and instructions to reply to the request of the user. The goal of this task is explicitly set to create a diverse database for the purpose of fine-tuning an LLM. Further instructions are given, such as paraphrasing, occasional spelling out of acronyms, brevity, correct spelling despite errors in the descriptions, etc.
- Structured output: same as in section III.C.
- Message: a single user message out of a list of 16 possible messages, such as "What is the functionality of [NAME_OF_NODE]?", "Why would I use [NAME_OF_NODE]?", "tell me a possible use case of [NAME_OF_NODE]", etc., where [NAME_OF_NODE] is replaced by the name of the function of which the description is included in the current system prompt.

This request is executed in a script iterating through the 16 user messages for each of the 21 function nodes in the KG. The process is repeated with a slightly different system prompt, resulting in 672 conversations each consisting of a single pair of question/instruction by the user and the corresponding response by the assistant. This database contains more than 180,000 content characters.

It is merged with the database of section III.C, i.e. the two JSONL files are concatenated. Furthermore, the lines of the concatenated file are shuffled in order to prevent artifacts related to ordering of this training data.

FT-KG step 3: fine-tuning Phi-3.5

Finally, the original base model is fine-tuned on the combined training data, i.e. on both the general GTlab information and the KG function node descriptions. The corresponding pipeline from step 3 of section III.C can be reused.

IV. Results

A. Fine-tuning GTlab information

The setup described in section III.C was found only after some iterations. Initial attempts at fine-tuning revealed an underwhelming performance of the model when answering questions about GTlab. After some attempts, effective conversations could be held with the resulting model. Representative examples of such conversations are listed in Table 1.

The responses of the assistant in conversations 1 and 2 are outright correct and concise. Conversation 3 contains a rather diffuse answer to a an admittedly imprecise question. The intent of the user was clarified in conversation 4 which lead to a more precise answer.

The training time for the final setup was approximately 1h 20min, which is a one-time computational price to pay for the permanent reduction of the required context window length. The number of parameters of the model was increased from initial 3,821,079,552 by 25,165,824; i.e. **the number of model parameters was increased by less than 0.66%**.

Two key factors enabled this expert-like assistant behavior of an isolated SLM:

1) large amounts of training data with verbally diverse and eloquent contents and
2) an elevated number of 10 training epochs.

B. Fine-tuning knowledge graph node descriptions

The fine-tuned model described in section III.E additionally contained the entire definitions of function nodes of the knowledge graph. As the tool definitions are rather short and the inflation stage also puts emphasis on brevity, the fine-tuning process consumed very similar computational resources and lead to an equal model size.

First, questions about the available tools were asked to the fine-tuned Phi. The example of the "Kontur_Plotter" tool as defined in section III.E is taken here. Representative examples are listed in Table 2 in order of increasing difficulty.

Conver. #	User	Assistant
1	What is GTlab?	GTlab is a comprehensive software tool for the design and analysis of gas turbine engines. It supports various types of engines, including civil, military, and aerospace applications.
2	How can I open a project in GTlab?	To open a project in GTlab, follow these steps: 1. Launch GTlab. 2. Navigate to the 'File' menu. 3. Select 'Open Project'. 4. Choose the project file you wish to open. This will load the project into GTlab for further analysis and design.
3	Tell me about workflow management in GTlab.	Workflow management in GTlab is a systematic approach to designing, analyzing, and optimizing processes. It involves creating a sequence of tasks that are interconnected, ensuring efficient and effective operation.
4	Tell me about workflow management and how processes are defined and executed in GTlab.	Workflow management in GTlab involves defining and executing processes that simulate the behavior of gas turbine engines. Processes are defined using a graphical interface, where nodes represent different components or steps, and edges represent the flow of data or materials between them. Each node can have inputs and outputs, and the execution follows the defined path, allowing for complex simulations of engine operations.

Table 1 Four examples of entire conversations with the fine-tuned version of Phi-3.5-mini-instruct. No additional context was required, i.e. no special prompting, no system messages, and no RAG was utilized.

One can see that specific questions are answered quite accurately, e.g. regarding the tool descriptions. However, a slight tendency towards "hallucination" can be noticed even in simple cases, e.g. pseudo-code when no programming language has been defined. In the third example, an open question is answered with general knowledge, hinting at the fact that the model was not overfitted even with an epoch count of 10.

To put this on a larger empirical basis, the fine-tuned model was also asked very general questions, like about commonly used Python packages or the past presidents of the United States of America. The model gave accurate responses for all general questions not related to GTlab or the KG data. A rigorous benchmarking of such general ability was not part of this case study.

The fine-tuned model's ability to select workflows from multiple possibilities was also investigated. The results are stated in the next subsection.

C. Knowledge graph retrieval-augmented generation

To evaluate the system's ability to generate workflows in scenarios with multiple possible paths, four benchmark workflows were defined. Each benchmark presents unique challenges, testing the system's capacity to handle redundancy, optional inputs, varying fidelity levels, and complex combinations of these factors. The benchmark workflows are shown in Figure 5. The principles of these examples apply equally well to other fields of computational engineering, even if the concrete examples all come from the context of structural engineering of jet engines.

Test Case A: Tool Redundancy

This benchmark examined the system's ability to resolve redundancy in tool selection. A stator geometry, referred to as "Stator 2," can be processed by two measurement tools, both capable of exporting data in identical formats (VMAP [21] or ANSYS). The subtle difference lies in the ease of use of the tools. The user query for this test case was: "Generate a PERMAS model for Stator 2." The system should correctly identify the "BladeMesher" tool as the optimal choice due to its better usability compared to the alternative.

Test Case B: Multi-Input Scenarios

In this benchmark, the system was tested on a workflow involving a finite element solver that requires a finite element model and a material specification as input. The material options included aluminum, steel, and titanium. The user query was: "I want a static stress analysis for Rotor 1 of the low-pressure compressor." The system should select

Conver. #	User	Assistant
1	Contour plotter	Generates a contour plot from a given dataset. Input: - Dataset: The data to be plotted, typically a matrix or a table. Output: - Contour Plot: A visual representation of the data showing contours of equal value. Example: `ContourPlot(dataMatrix)`
2	What are the inputs and the outputs of Kontur_Plotter?	The inputs for Kontur_Plotter are a mesh and a scalar field. The output is a contour plot. Inputs: - Mesh: A structured or unstructured mesh representing the domain of interest. - ScalarField: A scalar field defined on the mesh, which could be a result of a finite element analysis or any other numerical computation. Output: - ContourPlot: A visual representation of the scalar field values over the mesh, showing lines of constant value (contours).
3	Which tool can I use for visualization of displacement fields?	ParaView is a versatile tool that can be used for visualizing displacement fields. It supports various formats and provides a range of visualization options.

Table 2 Three examples of entire conversations with Phi-3.5-mini-instruct fine-tuned with additional KG data.

titanium as material, citing its suitability for low-pressure compressors due to its strength-to-weight ratio. Aluminum is generally too weak, while steel is too heavy for the application.

Test Case C: Multi-Fidelity Options

This test case involved designing a disk geometry for a low-pressure compressor. Three tools were available: an analytical design tool (fast but inaccurate), a disk optimization tool (accurate but slow), and a machine learning surrogate model (fast and accurate within its training range, cf. [22]). The user query was: "Generate a disk mesh for a low-pressure compressor." The system should select the machine learning surrogate model, as it provided a balanced solution suitable for the given context of low pressure compressors.

Test Case D: Complex Workflow Integration

The final benchmark combined the challenges of redundancy, multi-input options, and multi-fidelity tools. The query was: "Generate a Campbell diagram for Rotor 3 of the low-pressure compressor, name all necessary tools."

The benchmark queries were processed with the Graph RAG system shown in Figure 4 and the resulting sub-graph presented to a selection of different LLMs. The answer quality is visualized in Figure 6 for all test cases.

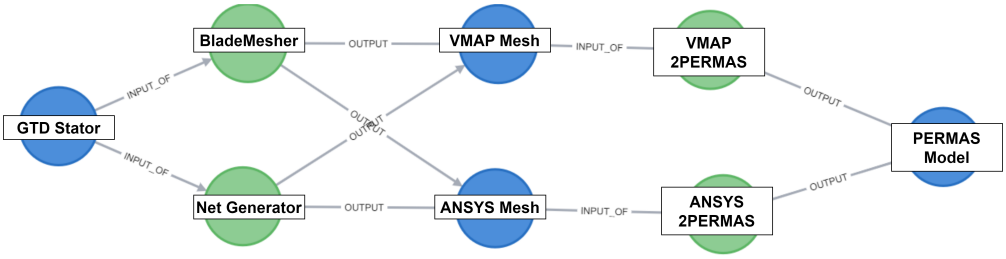
The results show that a certain level of "intelligence" is needed to solve the challenge of finding the most suitable path provided by the knowledge graph. The smallest tested model, Phi-3.5-mini both in its original and its fine-tuned configuration, failed to solve any testcase. In contrast, an also low-tier model like LLama 8B was able to give correct answers for testcases A and C. Starting from 70B parameters the models perform equally well as the latest ChatGPT model at the time of writing.

All models failed so solve the most complex testcase D. While the answer was technically feasible starting from models size 8B and higher, all models failed to identify the use of the surrogate model for disk generation in the complex scenario. Since this testcase works in isolation, an explanation could be the limited context window.

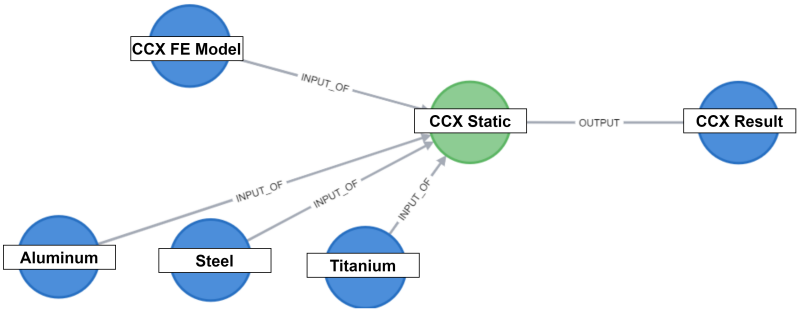
V. Conclusion and Outlook

This study demonstrated somewhat impressive capability of small models, SLMs, to gain expert knowledge about WfMS via fine-tuning. Transferring this finding to some of the specialized agents of part A could permanently reduce the required context window length with only moderate one-time computational investments. This would be a sustainable gain in efficiency and an easily scalable part of the multi-agent system.

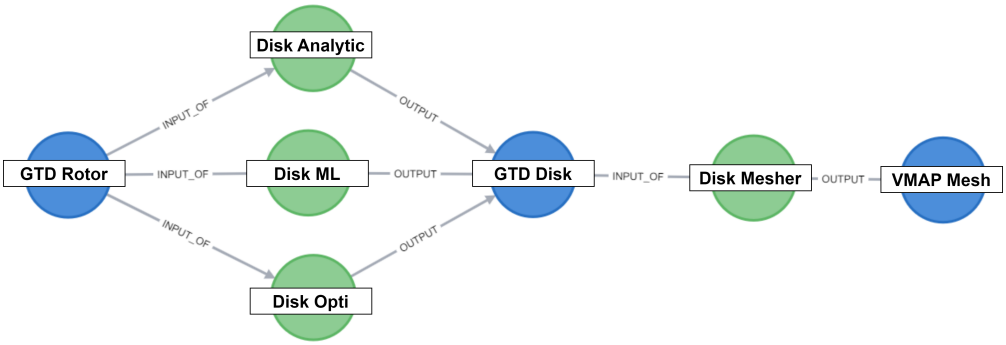
Moreover, the potential of LLMs in generating engineering workflows, even for complex, multi-disciplinary and non-unique problems was shown. Larger models, such as ChatGPT-4o but also Llama3.1-70B, exhibited strong performance on the test cases A, B, and C, yet struggled with the most complex test case D, showing the limitations in



Test Case A: Redundancy



Test Case B: Multi Input



Test Case C: Mult Fidelity

Fig. 5 System testcases for ambigous engineering workflows

logical reasoning and context window capacity. Smaller models, even when fine-tuned, were completely unable to solve the problem, underscoring the necessity for a minimum level of logical capability in addressing such tasks.

A key challenge identified was the context window limitation, particularly for complex workflows requiring the description of numerous tools from the benchmark knowledge graph. To address this, we propose integrating the multi-agent systems and graph-based approaches developed in part A of this series combined with both classical RAG and Graph RAG techniques.

An iterative pathfinding strategy is suggested as a promising solution. Instead of presenting the entire workflow graph to the LLM at once, such a step-by-step agent would evaluate each decision point sequentially, recognizing unsuitable paths early on. This approach could reduce the critical context window usage, especially when thinking about scaled-up context, such as very large and complex tool collections.

Future work will also investigate the potential of combining Graph Neural Networks (GNNs) and Transformers to address the challenges of generating workflows in multi-disciplinary engineering environments. GNNs could be employed to encode the structural relationships within the workflow graph, where nodes represent tools and edges and capture dependencies between them. This representation would allow the model to understand the interdependencies

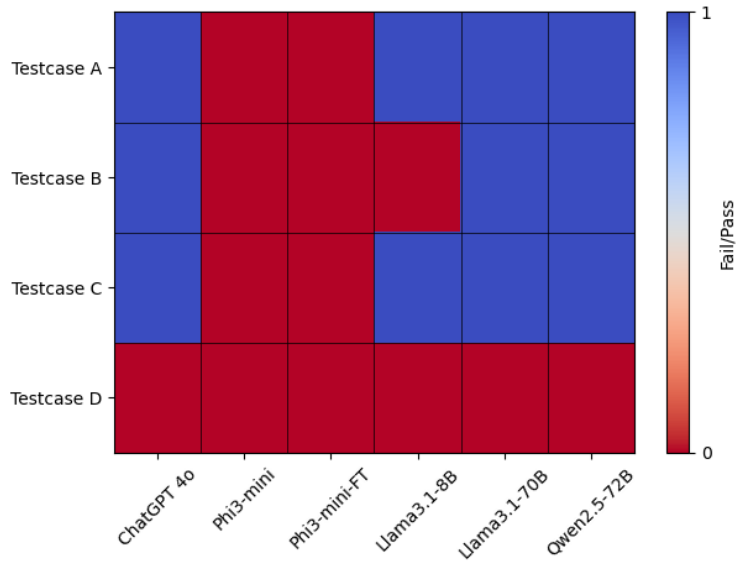


Fig. 6 Results for each testcase for different llms

between tools, such as how the output of a mesh generator influences subsequent finite element analysis. By propagating information across the graph, GNNs could provide a robust encoding of the workflow structure, ensuring that generated workflows respect tool compatibility and dependency constraints.

Acknowledgments

This work was supported by the Helmholtz Association's Initiative and Networking Fund on the HAICORE@FZJ partition.

Conflict of interest

The authors declare no conflict of interest related to this paper.

Contributions and originality

Contributions are quantitatively equal. Both authors contributed to every aspect of this work, with each author having different focal areas which are now listed. NF: conceptualization, methodology, KG, writing, proofreading. OK: fine-tuning, writing, proofreading.

Data availability

The data that support the findings of this study are available from the authors upon reasonable request.

References

[1] Peng, S., Kalliamvakou, E., Cihon, P., and Demirer, M., "The Impact of AI on Developer Productivity: Evidence from GitHub Copilot," arXiv, February 2023. URL `https://www.microsoft.com/en-us/research/publication/the-impact-of-ai-on-developer-productivity-evidence-from-github-copilot/`.

[2] Koziolek, H., Grüner, S., Hark, R., Ashiwal, V., Linsbauer, S., and Eskandani, N., "LLM-based and Retrieval-Augmented Control Code Generation," *Proceedings of the 1st International Workshop on Large Language Models for Code*, ACM, New York, NY, USA, 04202024, pp. 22–29. https://doi.org/10.1145/3643795.3648384.

[3] Wu, Y., Yue, T., Zhang, S., Wang, C., and Wu, Q., "StateFlow: Enhancing LLM Task-Solving through State-Driven Workflows," , 2024. https://doi.org/10.48550/ARXIV.2403.11322.

[4] Zeng, Z., Watson, W., Cho, N., Rahimi, S., Reynolds, S., Balch, T., and Veloso, M., "FlowMind: Automatic Workflow Generation with LLMs," *Proceedings of the Fourth ACM International Conference on AI in Finance*, Association for Computing Machinery, Erscheinungsort nicht ermittelbar, 2023, pp. 73–81. https://doi.org/10.1145/3604237.3626908.

[5] Pradas Gomez, A., Panarotto, M., and Isaksson, O., "Evaluation of Different Large Language Model Agent Frameworks for Design Engineering Tasks," *Proceedings of NordDesign 2024*, The Design Society, 2024, pp. 693–702. https://doi.org/10.35199/NORDDESIGN2024.74.

[6] Reitenbach, S., Siggel, M., and Bolemant, M., "Enhanced Workflow Management using an Artificial Intelligence ChatBot," *AIAA SCITECH 2024 Forum*, 2024. https://doi.org/10.2514/6.2024-0917, URL https://arc.aiaa.org/doi/abs/10.2514/6.2024-0917.

[7] Reitenbach, S., Siggel, M., and Bolemant, M., "Evolving AI-Driven Workflow Management, Part A: Strategies for Token Window Challenges and Utilization of Provenance Data," *AIAA SCITECH 2025 Forum*, 2025.

[8] Reitenbach, S., Vieweg, M., Becker, R., Hollmann, C., Wolters, F., Schmeink, J., Otten, T., and Siggel, M., "Collaborative Aircraft Engine Preliminary Design using a Virtual Engine Platform, Part A: Architecture and Methodology," *AIAA Scitech 2020 Forum*, 2020. https://doi.org/10.2514/6.2020-0867, URL https://arc.aiaa.org/doi/abs/10.2514/6.2020-0867.

[9] Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y., "HellaSwag: Can a Machine Really Finish Your Sentence?" *CoRR*, Vol. abs/1905.07830, 2019. URL http://arxiv.org/abs/1905.07830.

[10] Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al., "Program Synthesis with Large Language Models," *arXiv preprint arXiv:2108.07732*, 2021.

[11] Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J., "Training Verifiers to Solve Math Word Problems," *CoRR*, Vol. abs/2110.14168, 2021. URL https://arxiv.org/abs/2110.14168.

[12] Abdin, M., Aneja, J., Awadalla, H., Awadallah, A., Awan, A. A., Bach, N., Bahree, A., Bakhtiari, A., Bao, J., Behl, H., Benhaim, A., Bilenko, M., Bjorck, J., Bubeck, S., Cai, M., Cai, Q., Chaudhary, V., Chen, D., Chen, D., Chen, W., Chen, Y.-C., Chen, Y.-L., Cheng, H., Chopra, P., Dai, X., Dixon, M., Eldan, R., Fragoso, V., Gao, J., Gao, M., Gao, M., Garg, A., Del Giorno, A., Goswami, A., Gunasekar, S., Haider, E., Hao, J., Hewett, R. J., Hu, W., Huynh, J., Iter, D., Jacobs, S. A., Javaheripi, M., Jin, X., Karampatziakis, N., Kauffmann, P., Khademi, M., Kim, D., Kim, Y. J., Kurilenko, L., Lee, J. R., Lee, Y. T., Li, Y., Li, Y., Liang, C., Liden, L., Lin, X., Lin, Z., Liu, C., Liu, L., Liu, M., Liu, W., Liu, X., Luo, C., Madan, P., Mahmoudzadeh, A., Majercak, D., Mazzola, M., Mendes, C. C. T., Mitra, A., Modi, H., Nguyen, A., Norick, B., Patra, B., Perez-Becker, D., Portet, T., Pryzant, R., Qin, H., Radmilac, M., Ren, L., de Rosa, G., Rosset, C., Roy, S., Ruwase, O., Saarikivi, O., Saied, A., Salim, A., Santacroce, M., Shah, S., Shang, N., Sharma, H., Shen, Y., Shukla, S., Song, X., Tanaka, M., Tupini, A., Vaddamanu, P., Wang, C., Wang, G., Wang, L., Wang, S., Wang, X., Wang, Y., Ward, R., Wen, W., Witte, P., Wu, H., Wu, X., Wyatt, M., Xiao, B., Xu, C., Xu, J., Xu, W., Xue, J., Yadav, S., Yang, F., Yang, J., Yang, Y., Yang, Z., Yu, D., Yuan, L., Zhang, C., Zhang, C., Zhang, J., Zhang, L. L., Zhang, Y., Zhang, Y., Zhang, Y., and Zhou, X., "Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone," , 2024. https://doi.org/10.48550/ARXIV.2404.14219.

[13] Whitfield, D., "Using GPT-2 to Create Synthetic Data to Improve the Prediction Performance of NLP Machine Learning Classification Models," , 2021. https://doi.org/10.48550/ARXIV.2104.10658.

[14] Sufi, F., "Generative Pre-Trained Transformer (GPT) in Research: A Systematic Review on Data Augmentation," *Information*, Vol. 15, No. 2, 2024, p. 99. https://doi.org/10.3390/info15020099.

[15] Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., Drame, M., Lhoest, Q., and Rush, A. M., "Transformers: State-of-the-Art Natural Language Processing," Association for Computational Linguistics, 2020, pp. 38–45. URL https://www.aclweb.org/anthology/2020.emnlp-demos.6.

[16] Microsoft, "A simple example on using SFTTrainer and Accelerate to finetune Phi-3 models," https://huggingface.co/microsoft/Phi-3.5-mini-instruct/raw/af0dfb8029e8a74545d0736d30cb6b58d2f0f3f0/sample_finetune.py, 2024.

[17] Gugger, S., Debut, L., Wolf, T., Schmid, P., Mueller, Z., Mangrulkar, S., Sun, M., and Bossan, B., "Accelerate: Training and inference at scale made simple, efficient and adaptable." https://github.com/huggingface/accelerate, 2022.

[18] Rasley, J., Rajbhandari, S., Ruwase, O., and He, Y., "DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters," *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, Association for Computing Machinery, New York, NY, USA, 2020. https://doi.org/10.1145/3394486.3406703.

[19] Ansel, J., Yang, E., He, H., Gimelshein, N., Jain, A., Voznesensky, M., Bao, B., Bell, P., Berard, D., Burovski, E., Chauhan, G., Chourdia, A., Constable, W., Desmaison, A., DeVito, Z., Ellison, E., Feng, W., Gong, J., Gschwind, M., Hirsh, B., Huang, S., Kalambarkar, K., Kirsch, L., Lazos, M., Lezcano, M., Liang, Y., Liang, J., Lu, Y., Luk, C., Maher, B., Pan, Y., Puhrsch, C., Reso, M., Saroufim, M., Siraichi, M. Y., Suk, H., Suo, M., Tillet, P., Wang, E., Wang, X., Wen, W., Zhang, S., Zhao, X., Zhou, K., Zou, R., Mathews, A., Chanan, G., Wu, P., and Chintala, S., "PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation," *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*, ACM, 2024. https://doi.org/10.1145/3620665.3640366.

[20] Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., and Larson, J., "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," , 2024. URL https://arxiv.org/abs/2404.16130.

[21] "VMAP Standards Community e.V." https://www.vmap-standard.org, 2024.

[22] Forsthofer, N., "PhD Thesis: A Predictive Design Method for Compressor Blisks," *DLR Forschungsbericht*, 2024, p. 164. https://doi.org/10.57676/n1bj-4w35.